الواجب المنزلي 2 — هل ستتجاوز القذيفة الجدار؟

البرمجة العلمية للفيزيائيين | الأسبوع 2 | الموضوع الأساسي: الجمل الشرطية (`if`, `elseif`, `else`)

هدف التمرين

ستكتبون في هذا التمرين برنامجًا كاملًا بلغة Julia: يقرأ خمسة أعداد من لوحة المفاتيح، ويتحقق من استيفائها شروط النموذج، ويحسب ارتفاع القذيفة عند موضع الجدار، ثم يطبع النتيجة. أكملوا بأنفسكم أوامر الإدخال والحسابات والجمل الشرطية وأوامر الإخراج. يطبع البرنامج سطر نتيجة واحدًا أو ثلاثة أسطر، بحسب الحالة. لا حاجة إلى تعريف دوال أو كتابة حلقات.

النموذج الفيزيائي

تُطلق قذيفة من مستوى الأرض بسرعة ابتدائية `speed_m_s` وبزاوية `angle_deg` فوق الأفق. يقع جدار على مسافة أفقية `wall_x_m`، وارتفاعه `wall_height_m`. مقدار تسارع الجاذبية هو `gravity_m_s2`. نختار الاتجاه الأفقي الموجب نحو الجدار، والاتجاه الرأسي الموجب نحو الأعلى. نفترض أرضًا أفقية وجاذبية ثابتة ونهمل مقاومة الهواء.

في الصيغ التالية، v_0 هي سرعة الإطلاق، وθ هي زاوية الإطلاق، وg هو مقدار تسارع الجاذبية. احسبوا:

- السرعة الأفقية: $vx = v_0 \cos(\theta)$
- الزمن اللازم للوصول إلى مستوى الجدار: `t = wall_x_m / vx`
- الارتفاع المحسوب عند مستوى الجدار: $height_at_wall = v_0 \sin(\theta)\, t - g\, t^2 / 2$
- السرعة الرأسية عند مستوى الجدار: $vy = v_0 \sin(\theta) - g\, t$

يمكن استخدام `sind` و`cosd`، أو تحويل الزاوية إلى راديان باستخدام `deg2rad`.

يُحسب `height_at_wall` وفق الامتداد الرياضي للمسار، حتى إذا كانت القذيفة قد سقطت على الأرض قبل الجدار. يعني الارتفاع السالب أنها سقطت قبل الجدار. في هذه الحالة، اطبعوا رسالة السقوط فقط؛ لا حاجة إلى حساب السرعة الرأسية عند مستوى الجدار. ولا حاجة إلى محاكاة تصادم أو حساب زمن السقوط على نحو منفصل.

الإدخال

يقرأ البرنامج خمسة أعداد، عددًا واحدًا في كل سطر، بالترتيب التالي:

الترتيب	المتغير	المعنى	الوحدة
1	`speed_m_s`	سرعة الإطلاق	m/s
2	`angle_deg`	زاوية الإطلاق	درجات
3	`wall_x_m`	المسافة الأفقية إلى الجدار	m
4	`wall_height_m`	ارتفاع الجدار	m
5	`gravity_m_s2`	مقدار تسارع الجاذبية	m/s²

```
Launch speed (m/s):
Launch angle (deg):
Wall distance (m):
Wall height (m):
Gravity (m/s^2):
```

نجمع عمليتين لقراءة كل قيمة: تقرأ `readline()` سطرًا نصيًا، ثم تحوّله `parse(Float64, ...)` إلى عدد من النوع `Float64`. أوامر قراءة القيمة الأولى موجودة في الملف الهيكلي:

```
println("Launch speed (m/s):")
speed_m_s = parse(Float64, readline())
```

أكملوا أوامر إدخال القيم الأربع المتبقية بالطريقة نفسها. لكل قيمة، اطبعوا أولًا الرسالة الواردة في الجدول باستخدام `println`، ثم اقرؤوا العدد. يجب أن تظهر كل رسالة طلب إدخال في سطر منفصل.

يمكن الافتراض في هذا التمرين أن كل سطر يحتوي عددًا بصيغة تستطيع `parse` تحويلها. لا حاجة إلى معالجة نص مثل `hello`، أو أسطر مفقودة، أو إعادة طلب الإدخال بعد خطأ.

قيم الإدخال المسموح بها

يجب أن تكون القيم الخمس منتهية. بالإضافة إلى ذلك:

- `speed_m_s > 0`
- `0 <= angle_deg < 90`
- `wall_x_m >= 0`
- `wall_height_m >= 0`
- `gravity_m_s2 > 0`

قبل القسمة، تحققوا أيضًا من أن السرعة الأفقية موجبة: `0 < vx`.

بما أن الإدخال يُحوّل باستخدام `parse(Float64, ...)`، فإن كل قيمة تُقرأ بنجاح تكون من النوع `Float64`. استخدموا `isfinite` وتحققوا من المجالات المذكورة أعلاه. لا حاجة إلى فحوص إضافية للنوع، مثل `isa Real` أو التحقق مما إذا كانت القيمة من النوع `Bool`.

إذا كانت قيمة غير صالحة، أوقفوا البرنامج باستخدام `throw(ArgumentError("..."))` **قبل حساب زمن الوصول أو الارتفاع أو السرعة الرأسية، وقبل طباعة أي سطر نتيجة.** يُسمح بأن تكون رسائل طلب الإدخال قد طُبعت قبل الفحص. يمكنكم اختيار نص رسالة الخطأ.

متغيرات النتيجة

لكل إدخال صالح، احسبوا `height_at_wall`، وهو الارتفاع المحسوب عند موضع الجدار بالمتر.

إذا كان `height_at_wall < 0`، اطبعوا رسالة السقوط أدناه. في هذه الحالة لا يُطلب تعريف `vertical_state` أو `clears`.

أما عندما يكون `height_at_wall >= 0`، فاحسبوا أيضًا السرعة الرأسية `vy` وخزّنوا النتائج التالية:

- `vertical_state` — السلسلة النصية `"rising"` إذا كان `0 < vy`، وإلا `"falling"`. وفق اصطلاح التمرين، تُسند `"falling"` أيضًا عندما يكون `0 == vy`.
- `clears` — قيمة منطقية: `true` إذا كان `height_at_wall >= wall_height_m`، وإلا `false`. يشير هذا المتغير إلى أن الارتفاع المحسوب يبلغ ارتفاع الجدار على الأقل. لذلك تكون قيمته `true` أيضًا عند إصابة الحافة العليا بالضبط، مع أن لهذه الحالة رسالة إخراج مستقلة.

الإخراج

تسقط القذيفة قبل الجدار

إذا كان `height_at_wall < 0`، يطبع البرنامج **سطر نتيجة واحدًا فقط**:

```
Result: The projectile lands before the wall.
```

لا تطبعوا سطري `Height at wall` و`Vertical motion` في هذه الحالة.

تصل القذيفة إلى موضع الجدار بارتفاع غير سالب

إذا كان `height_at_wall >= 0`، يطبع البرنامج **ثلاثة أسطر نتيجة بالضبط**:

```
Height at wall: <number> m
Vertical motion: <state>
Result: <result>
```

اعرضوا في السطر الأول الارتفاع الذي حسبتموه. يُسمح بالتنسيق العددي المعتاد في Julia، ويُقبل فرق لا يتجاوز `0.0001` متر عن القيمة المتوقعة. يمكن تقريب القيمة المعروضة إلى أربع منازل عشرية، لكن يجب استخدام الارتفاع الأصلي قبل التقريب في جميع الشروط.

في السطر الثاني، يجب أن تكون `<state>` بالضبط `rising` أو `falling`.

يجب أن يطابق السطر الثالث إحدى الرسائل الثلاث التالية تمامًا:

الشرط	سطر الإخراج الثالث
`0 <= height_at_wall < wall_height_m`	`Result: The projectile hits the wall.`
`height_at_wall == wall_height_m`	`Result: The projectile hits the edge of the wall.`
`height_at_wall > wall_height_m`	`Result: The projectile clears the wall.`

استخدموا شرطًا خارجيًا للتمييز بين السقوط قبل الجدار والوصول إلى موضع الجدار بارتفاع غير سالب. داخل الفرع الثاني، استخدموا شروطًا متداخلة لتحديد الحالة الرأسية وتصنيف ما يحدث عند الجدار. ضعوا طباعة الارتفاع والحالة الرأسية داخل هذا الفرع فقط.

اطبعوا فقط رسائل طلب الإدخال وأسطر النتيجة المطلوبة للحالة. لا تطبعوا عناوين أو رسائل إضافية أو قيمًا وسيطة.

لفحص إصابة الحافة العليا بالضبط، استخدموا == على القيم قبل التقريب. تستخدم حالة التساوي في جدول الفحص الذاتي مسافة جدار صفرية وارتفاع جدار صفريًا، لذلك يتحقق فيها تساوٍ دقيق. لا حاجة إلى إضافة سماحية لفحص التساوي في هذا التمرين.

عند `speed_m_s = 20.0` و`angle_deg = 45.0` و`wall_x_m = 10.0` و`wall_height_m = 7.0` و`gravity_m_s2 = 9.81`، تكون أسطر النتيجة الثلاثة:

```
Height at wall: 7.5475 m
Vertical motion: rising
Result: The projectile clears the wall.
```

الفحص الذاتي

اختبروا البرنامج بالحالات الست التالية. استخدموا `gravity_m_s2 = 9.81` في جميع الحالات. يوضح عمودا الارتفاع والحالة الرأسية الإخراج المتوقع؛ والارتفاعات بالمتر. تشير الشرطة إلى أن السطر لا يُطبع. تشير أوصاف النتائج المختصرة إلى الرسائل الكاملة في قسم الإخراج.

`speed_m_s`	`angle_deg`	`wall_x_m`	`wall_height_m`	Height (m)	State	Result
20	45	10	7	7.5475	rising	clears
20	45	10	8	7.5475	rising	hits wall
20	45	30	7	7.9275	falling	clears
20	45	30	8	7.9275	falling	hits wall
20	45	50	2	—	—	lands before wall
20	0	0	0	0.0	falling	hits edge

في حالة السقوط قبل الجدار، يكون الارتفاع المحسوب نحو `11.3125` − متر. استخدموه لفحص الشرط، لكن لا تطبعوه. سطر النتيجة الوحيد في هذه الحالة هو رسالة السقوط.

تفحص الحالة الأخيرة التساوي الدقيق في الارتفاع. هي حالة مصطنعة فيزيائيًا، لكنها مفيدة لفحص أحد حدود الحالات.

افحصوا أيضًا مدخلات غير صالحة: سرعة صفرية أو سالبة، زاوية سالبة أو زاوية مقدارها `90` درجة أو أكثر، مسافة جدار سالبة، ارتفاع جدار سالب، تسارع جاذبية صفريًا أو سالبًا، وقيمة غير منتهية مثل `Inf` أو `NaN`. في كل تشغيل من هذه التشغيلات يجب أن يتوقف البرنامج بخطأ `ArgumentError` دون طباعة أي سطر نتيجة.

يمكن لطاقم التدريس فحص البرنامج بتزويده بخمسة أعداد بالترتيب نفسه المستخدم عند الإدخال من لوحة المفاتيح. يتجاهل فحص الإخراج رسائل طلب الإدخال، ويتوقع سطر نتيجة واحدًا عند السقوط قبل الجدار وثلاثة أسطر في سائر الحالات. تُقارن الارتفاعات المطبوعة تقريبيًا، وتُقارن رسائل الحالة والنتيجة بدقة. يجب أن يحسب الكود أيضًا `clears` وفق التعريف أعلاه داخل فرع الارتفاع غير السالب. ستستخدم الفحوص الإضافية المتطلبات ومجالات الإدخال المذكورة في هذه الورقة فقط.

العمل والتسليم

1. افتحوا `HW2_student_template.jl` في VS Code واحفظوا نسخة باسم `YOUR_ID.jl`.
2. أكملوا أوامر إدخال القيم الأربع الناقصة، ثم اكتبوا الحسابات والشروط وأوامر الإخراج بين `# === BEGIN STUDENT CODE ===` و`# === END STUDENT CODE ===`. استبدلوا سطر `error(...)` بحلكم، واتركوا سطري العلامتين دون تغيير.
3. افتحوا طرفية في المجلد الذي يحتوي الملف، وشغّلوا `julia YOUR_ID.jl`، ثم أدخلوا عددًا واحدًا في كل سطر. استبدلوا `YOUR_ID` برقم الطالب الخاص بكم وافحصوا الحالات أعلاه.
4. سلّموا **الملف كاملًا** في صفحة المقرر على Moodle.

لا حاجة إلى تثبيت حزم إضافية، أو قراءة ملفات من داخل البرنامج، أو تعريف دوال، أو كتابة حلقات.